02 / ALGEBRA / EX25

Python reference guide

NumPy fundamentals: representing, operating on, and measuring vectors.

DOCUMENT 02 / REFERENCE

What you will learn

- Create arrays and inspect their components.
- Use addition, subtraction, scaling, and norms with explicit syntax.
- Distinguish Python list behavior from NumPy array operations.

Prerequisites: Module imports, variables, and basic arithmetic.

Reading order: 1. Theory → 2. Reference → 3. Exercise → 4. Pseudocode → 5. Programming.

1. Import NumPy

NumPy is a numerical computing library. The conventional import alias is **np**. No configuration files or testing frameworks are necessary for this exercise.

```
import numpy as np
```

2. np.array: create a vector

Typical form: `np.array(data, dtype=float)`.

Parameter	Input / meaning
data	List, tuple, or other sequence of vector components.
dtype=float	Optional: request floating-point values.
Return value	A NumPy np.ndarray object.

```
v = np.array([5.0, -2.0])
print(v)        # [ 5. -2.]
print(v.shape)  # (2,)
```

The shape `(2,)` describes a 1D array of two elements, not a 2×1 matrix.

3. Components and dimensions

```
v = np.array([5.0, -2.0])
x = v[0]       # 5.0
y = v[1]       # -2.0
n = len(v)     # 2
```

Indices start at zero. Before adding vectors, make sure the number of components matches.

4. Componentwise operations

For NumPy arrays, +, -, and * operate componentwise:

```
u = np.array([2.0, 1.0])
v = np.array([-1.0, 3.0])
print(u + v)    # [1. 4.]
print(u - v)    # [ 3. -2.]
print(2 * u)    # [4. 2.]
```

Watch out: `[2,1] + [-1,3]` concatenates standard Python lists. NumPy arrays, by contrast, add their numerical components.

5. np.linalg.norm: vector magnitude

Useful signature: `np.linalg.norm(x, ord=None)`.

Argument	Meaning
x	Input vector as a NumPy array.
ord=None	For a vector, defaults to the 2-norm.
ord=1	Sum of absolute component values.
ord=np.inf	Largest absolute component value.
Return value	A floating-point scalar.

```
v = np.array([6.0, 8.0])
print(np.linalg.norm(v))             # 10.0
print(np.linalg.norm(v, ord=1))      # 14.0
print(np.linalg.norm(v, ord=np.inf)) # 8.0
```

`np.linalg.norm(v)` gives one scalar; `v**2` instead squares each component separately.

6. Printing values with units

Python does not automatically track engineering units. Document them clearly and keep units consistent.

```
value = 5.67891
print(f"Magnitude: {value:.3f} N")
# Magnitude: 5.679 N
```

The format `:.3f` prints three decimal places without changing the stored value.

7. Manual checks and tolerances

Floating-point values may exhibit rounding differences. Check results with an appropriate tolerance.

```
magnitude = np.linalg.norm(np.array([3., 4.]))
print(abs(magnitude - 5.0) < 1e-10)  # True
```

This simple check uses `abs` and a comparison; no testing framework is needed yet.

8. Common mistakes

- Forgetting `import numpy as np`.
- Using Python lists when componentwise addition is intended.
- Combining vectors with incompatible dimensions.
- Mixing up vector components and vector magnitude.
- Omitting units or mixing N, kN, m, etc.

9. Quick reference

Goal	Tool
Create a vector	np.array([a, b], dtype=float)
Add / subtract	u + v; u - v
Scale vector	factor * u
2-norm	np.linalg.norm(u)
1- / infinity norm	np.linalg.norm(u, ord=1 / np.inf)
Number of components	len(u); u.shape